GitHub e PlatformIO

Fabrício Ströher da Silva
fabricio.stroher@ifpr.edu.br

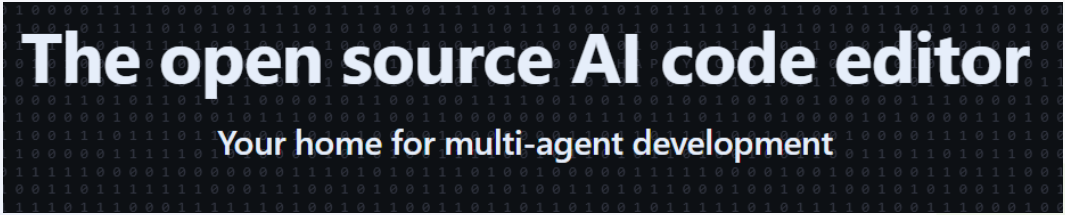
4 de novembro de 2026

Agenda

- Apresentação do ambiente de programação Visual Studio Code (VScode)
- Apresentação do platformIO
  - Projeto simples de monitor de temperatura com Arduino, LCD e sensores DS18B20
  - Simulação do projeto no Wokwi
  - Evolução do código: do monolítico ao modular
- GitHub

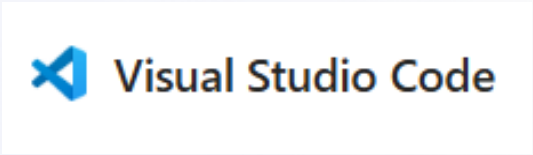
VScode

O que é VScode?



Primeiros passos

- Crie uma pasta, onde salvará o projeto
- Selecione esta pasta em *open folder*



Extensões

- Instale extensões considerando as necessidades:
  - PlatformIO IDE
  - C/C++ Extension Pack
  - Wokwi Simulador





PlatformIO

O que é PlatfomIO?



Your Gateway to Embedded Software Development Excellence

PlatformIO: estrutura típica do projeto

- `platformio.ini`: configuração do projeto
- `src/`: código-fonte principal
- `lib/`: bibliotecas locais
- `include/`: cabeçalhos
- `test/`: testes
- `.pio/`: artefatos de compilação



Dica: mantenha apenas o necessário no repositório; artefatos de build ficam no `.pio/`.

platformio.ini: exemplo comentado

Configuração mínima para Arduino

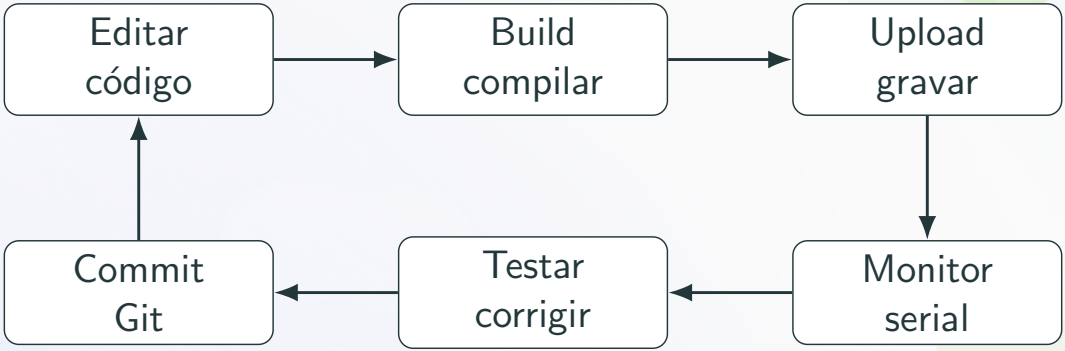
```
[env:uno]
platform = atmelavr
board = uno
framework = arduino
monitor_speed = 9600

lib_deps =
    paulstoffregen/OneWire
    milesburton/DallasTemperature
    marcoschwartz/LiquidCrystal_I2C
```

- `platform`: plataforma/microcontrolador
- `board`: placa usada
- `framework`: Arduino
- `lib_deps`: dependências gerenciadas pelo PlatformIO

▸ Documentação Oficial do platformio.ini

Ciclo de trabalho no PlatformIO



- Atalhos usuais: *Build*, *Upload*, *Monitor*
- O monitor serial auxilia na depuração
- Após validar, versionar com Git/GitHub

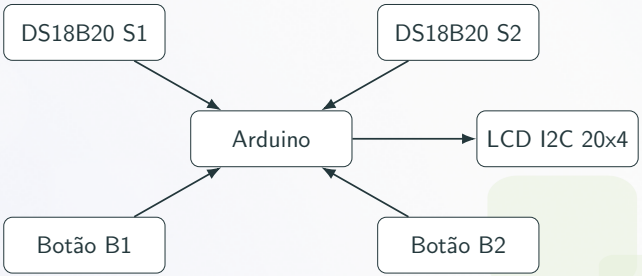
Estudo de caso do curso: monitor de temperatura

Hardware

- 2 sensores DS18B20 no barramento OneWire
- Display LCD I2C 20x4
- 2 botões com `INPUT_PULLUP`
- Arduino

Funcionalidades

- Menu de navegação
- Temperaturas em °C e °F
- Exibição dos endereços dos sensores
- Atualização não bloqueante dos sensores



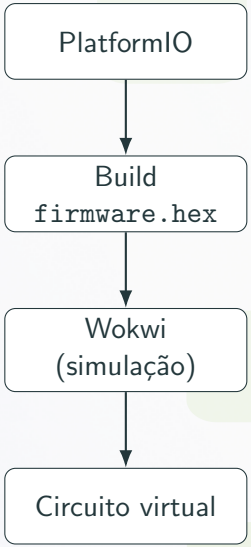
Wokwi: simulação de circuitos no VS Code

- Simulador de eletrônica e microcontroladores
- Extensão oficial: **Wokwi Simulador**
- Integra-se ao PlatformIO e outros toolchains
- Simula o firmware compilado, sem hardware físico
- Ideal para aulas, prototipagem e depuração

No curso

O mesmo projeto do monitor de temperatura será executado no Wokwi, permitindo testar LCD, DS18B20 e botões antes da montagem real.

▸ Documentação oficial do Wokwi



Arquivos de configuração do Wokwi

`wokwi.toml`

- Diz ao Wokwi onde está o firmware compilado
- Fica na raiz do projeto, ao lado do `platformio.ini`

```
[wokwi]
version = 1
firmware = '.pio/build/uno/firmware.hex'
elf = '.pio/build/uno/firmware.elf'
```

`diagram.json`

- Descreve os componentes e as conexões do circuito
- Pode ser criado no editor online do Wokwi ou manualmente
- Validação: `wokwi-cli lint`

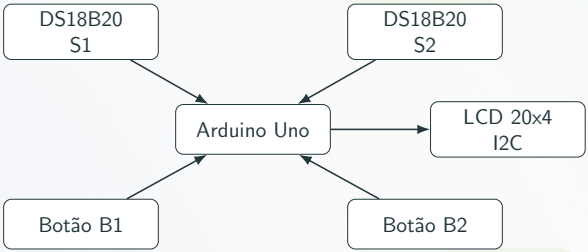
```
{
  "version": 1,
  "editor": "wokwi",
  "parts": [ ... ],
  "connections": [ ... ]
}
```

No projeto do curso: `wokwi.toml` aponta para `.pio/build/uno/firmware.hex`; `diagram.json` contém o Arduino Uno, 2 DS18B20, LCD 20x4 I2C, 2 botões e resistor de 4,7 kΩ.

Circuito do monitor de temperatura no Wokwi

Componentes no `diagram.json`

- `wokwi-arduino-uno`
- 2 × `wokwi-ds18b20` (endereços `FFCE86861604` e `FFEBDD521604`)
- `wokwi-lcd2004` com interface I2C
- 2 × `wokwi-pushbutton-6mm` (B1 e B2)
- `wokwi-resistor` de 4,7 kΩ (pull-up do OneWire)



- Conexões principais: DS18B20 no pino digital 9 (OneWire); LCD em A4 (SDA) e A5 (SCL); botões nos pinos 10 e 11.

Executando a simulação no VS Code

1. Compile o projeto no PlatformIO (botão *Build* ou `pio run`)
2. Verifique se `wokwi.toml` e `diagram.json` estão na raiz do projeto
3. Pressione `F1` e selecione **Wokwi: Start Simulator**

Fluxo de trabalho

```
# 1. compilar
pio run

# 2. abrir a paleta de comandos (F1) e iniciar:
#    Wokwi: Start Simulator
```

- O Wokwi recarrega automaticamente o novo firmware a cada *build*
- Sensores DS18B20 podem ser ajustados na simulação (temperatura)
- Botões virtuais permitem testar a navegação entre telas
- Monitor serial do Wokwi auxilia na depuração

Evolução do código: do monolítico ao modular

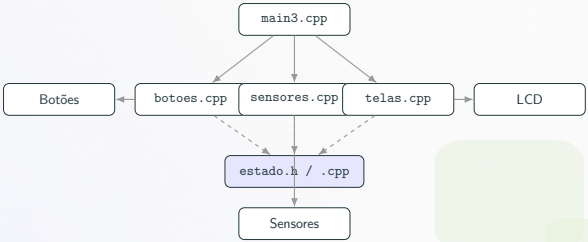
Versão	Arquivo	Característica
Monolítica	`main.cpp`	Tudo no `loop()`: botões, sensores, telas e estados
Com funções	`main2.cpp`	Separação em funções: `lerBotoes`, `atualizarSensores`, `desenharTela`
Modular	`main3.cpp`	Módulos: `botoes`, `sensores`, `telas`, `estado`
Orientada a objetos	`main4.cpp`	Classes: `Botao`, `SensorTemperatura`, `GerenciadorSensores`, `MaquinaTelas`

- A evolução reduz acoplamento e facilita manutenção
- Cada etapa é um commit didático no GitHub
- O aluno compara as abordagens no mesmo projeto

Arquitetura Modular (com funções) do exemplo

Organização

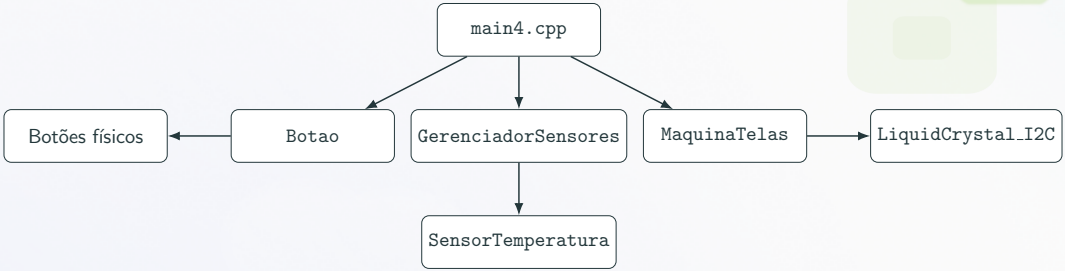
- `main3.cpp`: Apenas orquestra as chamadas no `loop()`
- `estado.h`: Centraliza as variáveis globais (sensores, telas, botões)
- `botoes.cpp`: Leitura física dos pinos
- `sensores.cpp`: Controle do barramento OneWire
- `telas.cpp`: Lógica de navegação e desenho no LCD



Característica

O código é mais limpo que o monolítico, mas ainda compartilha o estado global (C-style).

Arquitetura modular do exemplo



- `Botao`: leitura com detecção de borda
- `SensorTemperatura`: armazena endereço e temperaturas
- `GerenciadorSensores`: conversão não bloqueante
- `MaquinaTelas`: navegação e desenho no LCD

Boas práticas de refatoração observadas

- Separar responsabilidades: hardware, sensores, interface e navegação
- Evitar `delay()` longo; usar `millis()` para temporização
- Usar conversão não bloqueante: `setWaitForConversion(false)`
- Encapsular endereços e leituras em classes
- Reutilizar funções de desenho e impressão de endereço
- Versionar cada etapa com mensagens de commit claras
- Manter o `loop()` curto e legível

GitHub

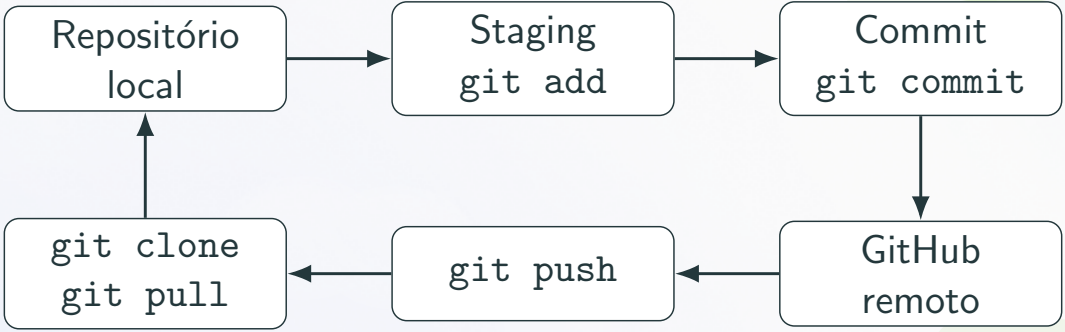
GitHub: conceitos essenciais

- **Repositório**: projeto versionado
- **Commit**: registro de alterações
- **Branch**: linha de desenvolvimento
- **Remote**: repositório remoto no GitHub
- **Push/Pull**: envio e recebimento de alterações
- **Issue**: registro de tarefa ou problema
- **Pull Request**: proposta de integração de mudanças

No curso

Cada versão do projeto de temperatura pode ser um commit ou uma branch, documentando a evolução do código.

Fluxo básico Git + GitHub




```
git init
git add .
git commit -m "Projeto inicial do monitor de temperatura"
git branch -M main
git remote add origin https://github.com/usuario/repositorio.git
git push -u origin main
```

Integração PlatformIO + GitHub

- Versionar código-fonte, `platformio.ini`, bibliotecas locais e documentação
- Ignorar artefatos de compilação e arquivos locais:

`.gitignore` **sugerido**

```
.pio/
.vscode/
*.log
```

- Criar `README.md` com:
  - descrição do projeto
  - hardware utilizado
  - bibliotecas e como compilar
  - imagens ou diagrama de ligação
  - versões/etapas didáticas
- Usar commits pequenos e descritivos

Atividade

Exercício sugerido

Escolha uma das versões (`main.cpp`, `main2.cpp`, `main3.cpp` ou `main4.cpp`), explique sua arquitetura e proponha uma melhoria.

Checklist final do curso

Ambiente

- VS Code instalado
- Extensões PlatformIO e C/C++
- Projeto PlatformIO criado
- Placa e monitor configurados

Projeto e versionamento

- Código compilando e gravando
- LCD e sensores funcionando
- Refatoração compreendida
- Repositório no GitHub
- `README.md` e `.gitignore`

Próximos passos

Criar uma branch para cada melhoria, abrir issues para tarefas e usar Pull Requests para integrar alterações.

Referências e encerramento

- Documentação do PlatformIO
- Documentação do Git e GitHub
- Bibliotecas: OneWire, DallasTemperature e LiquidCrystal_I2C
- Códigos do curso: `main.cpp`, `main2.cpp`, `main3.cpp`, `main4.cpp`

Dúvidas?